

VSE-IOT CORE ADAPTER V1.0.0

DETAIL SPECIFICATION

DETAIL SPEZIFIKATION

© 2021 ifm diagnostic gmbh

Meta Data

Document Management Information

Document	VSE-Iot core adapter Detail Specification
Project	Klicken oder tippen Sie hier, um Text einzugeben.
Ticket no.	Klicken oder tippen Sie hier, um Text einzugeben.
Version	0.42
Status	Draft
Date	20.04.2021
Author	Gavrailov, Angel

Document History

Version	Short description of change	Author	Reviewed	Approved
0.1	Initial version	Gavrailov, Angel/ 20.04.2021	Name/ date	Name/ date
0.2	Changes for 0.8.2	Gavrailov, Angel/ 26. Mar 2021	Name/ date	Name/ date
0.3	Changes for 1.0.0	Gavrailov, Angel/ 07 April 2021		
0.4	Added Inputs, Variants	Gavrailov, Angel/ 07 April 2021		
0.41	Updated Inputs, added Notations	Gavrailov, Angel/ 14 April 2021		
0.42	Removed Industrial Eth nodes	Gavrailov, Angel/ 20 April 2021		

Summary

The scope of this detail specification is the structure and functionality of the VSE-IoT Core Adapter.

Content

Meta Data 2
Document Management Information 2
Document History 2
Summary 2
1 Introduction 5
1.1 Rationale 5
1.2 Purpose of VSE-IoT Core adapter 5
1.3 System architecture 5
1.4 General remarks 5
1.4.1 Notations 5
1.4.1.1 Tree tables 5
2 ifm IoT-Core introduction 7
2.1 Purpose 7
2.2 Features 7
2.2.1 To the world outside 7
2.2.2 To the inner (device) world 7
2.3 Architecture 8
2.4 Basic IoT Core usage samples 8
3 Prerequisites 10
3.1 Infrastructure 10
3.2 Libraries 10
4 Tree specification 11
4.1 Access mechanisms 11
4.1.1 Polling access 11
4.1.2 Event based access 11
~~4.1.2.1 Time-based event generation~~ 11
4.1.2.2 Data change event generation (From v1.1) 12
4.2 Root node 12
4.2.1 Base path 12
4.2.2 Structure 12
4.2.3 Complex data structure description 12
4.2.4 Service description 12
4.3 System node 12
4.3.1 Base path 12
4.3.2 Structure 13
4.3.3 Complex data structure description 13
4.3.4 Service description 13

4.4 Device node 14
4.4.1 Structure 14
4.4.2 Complex data structure description 14
4.4.2.1 Status 14
4.4.2.2 Information 15
4.4.2.3 Documentation 16
4.4.2.4 Objects 16
4.4.2.5 Counters 19
4.4.2.6 Inputs 20
* Value node exists only for the “external” inputs and has r/w access 20
4.4.2.7 Variants 20
4.4.2.8 Alarms 20
4.4.2.9 RawData 22

1 Introduction

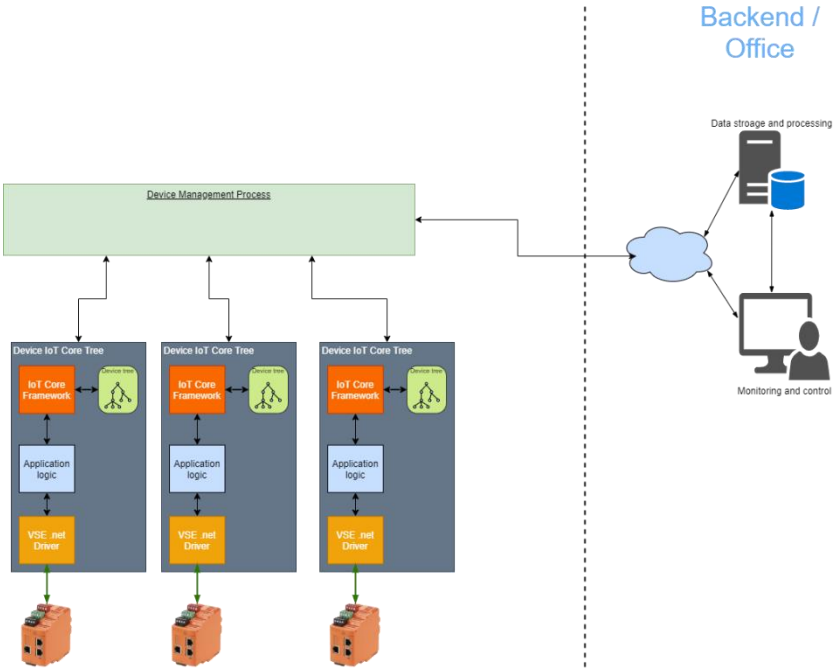
1.1 Rationale

This document specifies the concept of the VSE-IoT Core adapter, along with its feature and usage.

1.2 Purpose of VSE-IoT Core adapter

The purpose of the VSE-IoT Core adapter (from now on the Adapter) is to enable VSE devices to support the ifm's IoT-Core communication and thus enable their integration in the IT cloud infrastructure of ifm .

1.3 System architecture



- In this variant, the IoT Core Adapter System manager runs in external process, device management process (DMP)
- The DMP is responsible for starting, stopping and parametrization of the IoT Core Processes
- Each of the Devices' IoT Core Trees will be mirrored in the (external) main IoT Core tree
- Each request or response from a Device results in a doubled HTTP communication:
    - Client->Main Tree->Device Tree->Main Tree->Client

1.4 General remarks

1.4.1 Notations

1.4.1.1 Tree tables

- *JSON as a Value type*: means, that the value of the complete node and all subnodes can be read at once, using *getdata* service

- *NaN* - Not a Number. Usually this value appears, when no value has been received from the device or not enough data was available to perform certain calculation
- *ro* – the data in this field is Read-Only
- *rw* – The data in this field can be changed, using the *setdata* service

2 ifm IoT-Core introduction

2.1 Purpose

The idea behind the ifm IoT-Core (IoT-Core from now on) is to provide a unified mechanism for communication and interaction between:

- devices
- gateways and the devices behind them
- cloud-based services and devices and/or gateways

2.2 Features

2.2.1 To the world outside

- Representation of a device or a system as a Tree-structure
- Supports mapping of a tree in anther tree, thus building complex structures
- Supports standard elements of the tree nodes in order to simplify the usage
  - Data
  - Service
  - Event
- Supports standard profiles and custom ones
  - Process data
  - parameter
  - firmware update
- Protocols
  - HTTP(REST-Based)
  - MQTT (WIP)
  - OPC UA (WIP)

2.2.2 To the inner (device) world

The IoT-Core has two implementations

- c-based

Here the tree structure is defined during the compile-time and cannot vary dynamically during the device's operation. Only the values of the nodes can be changed internally or trough services, contacted from outside. Suitable for restricted devices or bootloaders, for example.

- .net-based

That is the implementation, which supports the full-feature catalog, dynamic creation and deletion of nodes, tree structure changes during the runtime and so on. It is suitable for devices, which can run .net core.

2.3 Architecture

How the IoT-Core represents the device

2.4 Basic IoT Core usage samples

Below are some samples how to interact with an IoT-Core enabled device/gateway:

- invoke service (get data element)

```
GET http://127.0.0.1:8091 HTTP/1.1
content-type: application/json

{
    "cid" : 0, "code":"request", "adr" : "/Device01/Objects/Object1/Warning/getdata",
    "cid":156
}
```

With this request we request the other device's IoT-Core to send us the value of the */Device01/Objects/Object1/Warning/getdata* data element (*getdata* is a standard service for data elements)

In a normal situation, the response should look like:

```
{
    "code":200,
    "adr":"/Device01/Objects/Object1/Warning/getdata",
    "cid":156,
    "data":"13.787"
}
```

The above means, that the data node has a value of *false*

- invoke service (set data element)

```
POST http://127.0.0.1:8091 HTTP/1.1
content-type: application/json

 {
    "cid" : 0, "code": 0, "adr" : "/Device01/Objects/Object1/Warning/setdata",
    "data": {
        "newvalue": 12.878
    }
}
```

In this case, we invoke the service *setdata*, which is a standard service for a data element (which is */Device01/Objects/Object1/Warning/setdata*), pass the new value (*12.878*) as a parameter.

Effectively, the above means, that we set the value of the *rawdata* element to *true*.

- subscribe for an event

Assume that the device IoT-Core listens on *http://127.0.0.1:8091* and we have another IoT-Core device or system under *http://127.0.0.1:9091*

```
POST http://127.0.0.1:8091 HTTP/1.1
content-type: application/json

 {
    "cid" : 0, "code": 0, "adr" : "/objects/08/runtime/windowed/new-data/subscribe",
    "data": {
        "callback" : "http://127.0.0.1:9091/remote/objects/newdata",
        "datatosend" : ["/objects/08/runtime/windowed"]
    }
}
```

What we do:

- subscribe to an event under */objects/08/runtime/windowed/new-data/subscribe*

  This event is generated from the device, implementing IoT-Core, when a data collection time-window has expired, and a new data is available

- Declare, that upon the occurrence of this event, we want the data under */objects/08/runtime/windowed* to be delivered to *http://127.0.0.1:9091/remote/objects/newdata*. That means, that upon an event, the source IoT-Core wraps the requested data sends it to the subscriber trough a simple http-POST to *http://127.0.0.1:9091/*

3 Prerequisites

3.1 Infrastructure

3.2 Libraries

The Adapter relies on the following libraries:

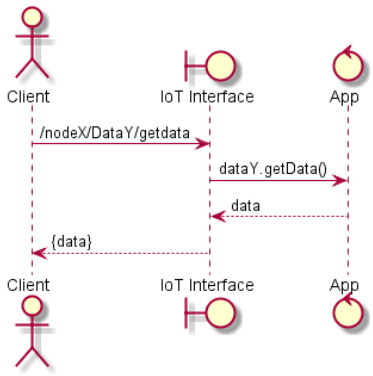
- .net core 3.1
- IFM SDK (VSE .net driver) v1.3.0
- Ifm IoT Core Library v1.2

4 Tree specification

4.1 Access mechanisms

4.1.1 Polling access

Using the polling access the client can read arbitrary data in every given moment. There are complex nodes, that should only be read as a block of data in order to be able to get a consistent data, because the data could change between to access.

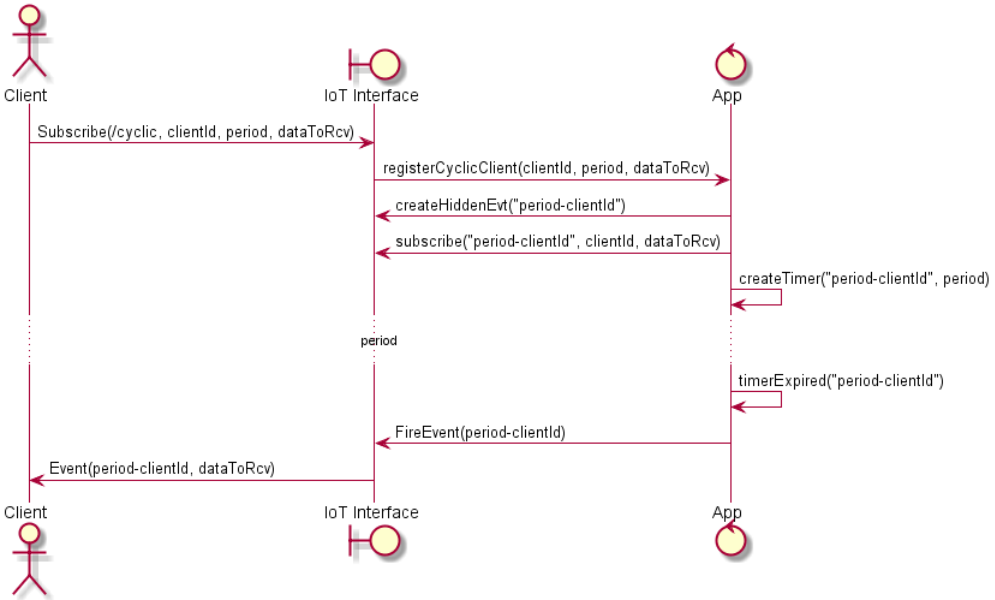


4.1.2 Event based access

In this case the data will be delivered to the client if event occurs, for which the client is registered. Following scenarios are foreseen to be implemented from the Adapter:

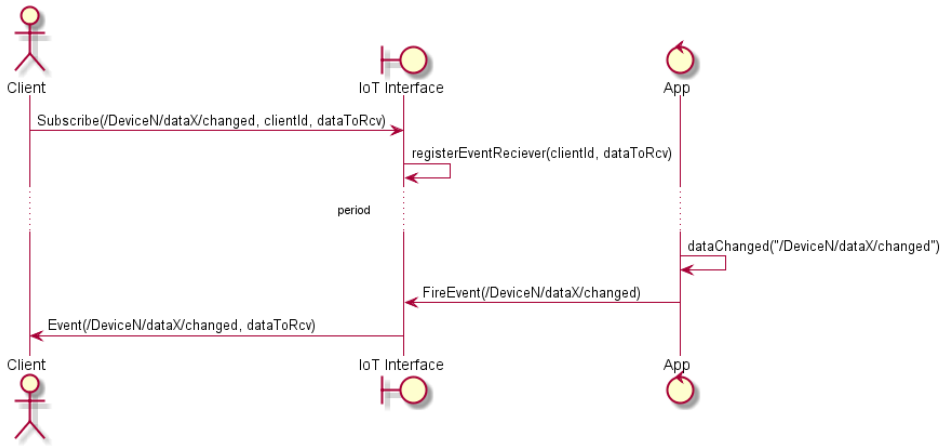
~~4.1.2.1 Time-based event generation~~

~~This is the case, where the client(s) require certain data or datasets to be delivered with a certain rate. In this case, the events are present on the tree top-level, the client can specify which exactly data from which device(s) should be delivered at event occurrence.~~



4.1.2.2 Data change event generation (From v1.1)

This is the case, where the client(s) require to be informed for changes in the values in certain data nodes. Here the events are implemented as per particular data node. The client has to subscribe for each data or event of interest separately.



4.2 Root node

4.2.1 Base path

The base path is /

4.2.2 Structure

Path	Type	Value type	Description
..System	structure	-	See 4.3
..Device	structure	-	See 4.4

4.2.3 Complex data structure description

4.2.4 Service description

4.3 System node

The *System Node* makes information and service available, which relate to the system, on which the IoT core instance is running

4.3.1 Base path

The base path is /**System**

4.3.2 Structure

Path	Type	Value type	Description	Format	Access
..DateTime	data	JSON	See members below	JSON Object	ro
..DateTime/Date	string	string	Current system date*	"2020-10-30"	ro
..DateTime/Time	string	String	Current system time*	"15:00:43"	ro
..DateTime/TimeZone	int	int	System timezone*	UTC Timezone	ro
..Version	data	JSON	See members below	JSON Object	ro
..Version/Server	data	string	Adapter version	"1.00.04"	ro
..Version/Driver	data	string	VSE Driver version	"1.3.0"	ro

* That is the host's date and time, i.e. those of the server, running the IoTCore Adapter. For Device DateTime, see 4.4.2.2.1

4.3.3 Complex data structure description

4.3.4 Service description

4.4 Device node

Since the IoT core shall support only one connected device, the *DeviceNode* gets instantiated only once. It holds the entire information, related to the connected device or only basic information, if the device cannot be connected or is in mode, other than *Supervise* or *Monitoring*.

4.4.1 Structure

Base path: **/Device**

Path	Type	Value type	Description	Format	Access
..Status	data	JSON	Device status information	See 4.4.2.1	ro
..Information	data	JSON	Generic device information	See 4.4.2.2	ro
..Documentation	data	JSON	Location/Client information	See 0	ro
..Objects	data	JSON	Object information	See 4.4.2.4	ro
..Counters	data	JSON	Counters information	See 0	ro
..Alarms	data	JSON	Alarms information	See 4.4.2.8	ro
..RawData	structure	-	RawData control node		

4.4.2 Complex data structure description

4.4.2.1 Status

Path	Type	Value type	Description	Format	Access
..Status/ConnectionState	data	string	Current connection state, enum	“connected”, “connecting”, “disconnected”	ro
..Status/SystemMode	data	string	Current system mode, enum	"none", "selftest, "setup”, "startup", “supervise", "measure"	ro
..Status/Error	data	string	Error description, in case of any	“wrong configuration”	ro
..Status/Changed	event	-	Invoked in case of device status change	-	-

“*Changed”* event is generated in case of:

- newly connected device
- device disconnected
- device reset
- system mode change

~~In any of the above situations, the device tree is generated again from scratch in order to avoid inconsistency.~~

4.4.2.2 Information

Path	Type	Value type	Description	Format	Access
..Information/Device	data	JSON	Device related data	See 4.4.2.2.1	ro
..Information/Status	data	JSON	Device state data	See 4.4.2.2.2	ro
..Information/IP	data	JSON	Network data	See 4.4.2.2.3	ro

4.4.2.2.1 Device data

Path	Type	Value type	Description	Format	Access
..Information/Device	data	JSON	See members below	JSON Object	ro
..Information/Device/Name	data	string	Device name	“Device Name”	ro
..Information/Device/Type	data	string	Device type	“VSE150”	ro
..Information/Device/Revision	data	string	Device revision	“AA”	ro
..Information/Device/Serial	data	string	Device serial number	“1 1 123456”	ro
..Information/Device/Firmware	data	string	Device firmware version	“V2.00.08”	Ro
..Information/Device/DateTime	data	JSON	See members below	JSON Object	ro
..Information/Device/ DateTime/Date	string	string	Current device date	"2020-10-30"	ro
..Information/Device/ DateTime/Time	string	String	Current device time, Normally UTC, depends however on the device configuration	"15:00:43"	ro

4.4.2.2.2 Device state:

Path	Type	Value type	Description	Format	Access
..Information/Status	data	JSON	See members below	JSON Object	ro
..Information/Status/SystemMode	data	string	System mode, enum	“Measure”, “Self test”, “Supervise, “Setup”	ro
..Information/Status/ActiveVariant	data	string	Device type	“VSE150”	ro

4.4.2.2.3 Network information

Path	Type	Value type	Description	Format	Access
..Information/IP	data	JSON	See members below	JSON Object	ro
..Information/IP/HostName	data	string	System mode, enum	“Hostname-1”	ro
..Information/IP/DHCP	data	bool	DHCP Mode on	true, false	ro
..Information/IP/Address	data	string	IP Address	“192.168.1.13”	ro
..Information/IP/Port	data	int	Connection port	12345	ro
..Information/IP/Gateway	data	string	Gateway IP Address	“192.168.1.1”	ro

..Information/IP/Submask	data	string	Network submask	“255.255.255.0”	ro
..Information/IP/MACAddress	data	string	Device MAC Address	“de:ad:be:ef:00”	ro

4.4.2.3 Documentation

Path	Type	Value type	Description	Format	Access
..Documentation	data	JSON	See members below	JSON Object	ro
.. Documentation /Company	data	string	Company name	“ifm diagnostic”	ro
.. Documentation /Address	data	string	Company address	“Aicher Str. 1”	ro
.. Documentation /City	data	string	Company City	“Rosenheim”	ro
.. Documentation /Location	data	string	Device location	“Machine room”	ro
.. Documentation /Machine	data	string	Machine information	“DMG DUO ”	ro
.. Documentation /CreatedBy	data	string	Configuration Author	“Max M.”	ro
.. Documentation /CreationDate	data	string	Configuration Creation date	“2020-04-12”	ro
.. Documentation /LastChange	data	string	Configuration last change date	“2020-04-18”	ro
.. Documentation /Description	data	string	Application description	“Balance”	ro

4.4.2.4 Objects

Under this node appear the only the currently configured objects. “N” is the Id of the corresponding object (1..24).

Path	Type	Value type	Description	Format	Access
..Objects/ObjectN	data	JSON	Data for ObjectN	See 4.4.2.4.1	ro

The data of *ObjectN* is specified in the table below.

4.4.2.4.1 ObjectN

Path	Type	Value type	Description	Format	Access
..ObjectN	data	JSON	See members below	JSON Object	ro
.. ObjectN/Name	data	string	Configured object name	“SE01_Unbalance_01”	ro
.. ObjectN/Type	data	string	Object type, enum	"unbalance", "bearing", "a_rms_fd", "v_rms_fd", "d_rms_fd", "a_rms_td",	ro

				"v_rms_rd", "a_peak_td"	
.. ObjectN/ID	data	int	Object ID	1	ro
.. ObjectN/Value1	data	float	Value according to the configuration. Dependent on the object type could be Min, Max, Mean and gets updated at the end of the averaging period. (1)	5.6789	ro
.. ObjectN/Value2	data	float	Value according to the configuration. Dependent on the object type could be Min, Max, Mean and gets updated at the end of the averaging period. (1)	5.6789	ro
.. ObjectN/Unit	data	string	Value unit	“mm/s”	ro
.. ObjectN/State	data	string	Current object state, enum	“ok”, “error”, “inactive-by-variant”	ro
.. ObjectN/StateEvent	event	--	Occurs, when the State changes	--	--
.. ObjectN /Error	data	string	Error, combination of enums, divided by "|”	"internal-error", "calc-failed", "rms-outside", "rpm-instable", "no-teach-in", "no-teach-reference-1", "no-teach-reference-2", "disabled-by-emphasis-1", "disabled-by-emphasis-2", "is-warning", "is-damage "	ro
.. ObjectN /ErrorEvent	event	--	Occurs, when the value of the Error field changes	--	--
.. ObjectN/RotSpeed	data	float	Rotational speed, calculated using the method for Value1 (1)	2.1345	ro

.. ObjectN/RefValue	data	float	RefValue, calculated using the method for Value1 (1)	12.789	ro
.. ObjectN/Warning	data	float	Current warn level	12.789	ro
.. ObjectN/BaseLine	data	float	Current baseline	12.789	
.. ObjectN/Damage	data	float	Current damage level	35.658	ro
.. ObjectN/RotValueID	data	int	ID of the assigned Rotation value	1	ro
.. ObjectN/RefValueID	data	Int	ID of the assigned Reference value	2	ro
.. ObjectN/InputID	data	Int	ID of the assigned input	3	ro
.. ObjectN/InputType	data	string	Type of assigned Input		
.. ObjectN/AveragingPeriod	data	Int	Averaging period for the *Value* field in milliseconds	1000	Rw
.. ObjectN/DataAge	data	int	Period in ms, since the last data arrival	12334	R
.. ObjectN/ValueFrequency	data	float	Average frequency over 1000ms, at which object values arrive, in Hz	123.45	R
.. ObjectN/DynamicDataMonitoring	data	bool	If true, the object monitoring is active. Default is true, except for fast objects[2]	True/false	R/W

~~**Major issue:** *ErrorEvent* and *StateEvent* can occur inside an averaging period. If one subscribes for those events, there is no way to deliver the *Value*, *RotSpeed*, *Baseline* at which the state changed, or the error occurred. In order to be able to do this, those instant values must be present in the tree as nodes, different from the averaged ones.~~

[1] Object value calculation

The below “Mean”, “Max” and “Mean” values are those, calculated from the gathered data set, i.e. data during the last *AveragingPeriod* milliseconds. If no new Object Data was available in this period, those values below represent the last available Object Data sample.

Object type	Type Value1	Type Value2
Unbalance	Mean	Max
Bearing	Mean	Max
A_RMS_FD	Mean	Max
V_RMS_FD	Mean	Max
D_RMS_FD	Mean	Max
A_RMS_TD	Mean	Max
V_RMS_TD	Mean	Max
A_PEAK_TD	Max	---

UpperMonitor	Max	---
LowerMonitor	Min	---
Other	Mean	Max

[2] Monitoring of fast objects is not turned on at startup. One can do it later, however this will almost for sure block the rest of the monitoring, since the values from such objects tend to occupy most of the communication channel.

4.4.2.5 Counters

Under this node appear all the counters, that the device supports. “N” in the table below.

Path	Type	Value type	Description	Format	Access
..Counters/CounterN	data	JSON	Data for CounterN	See	ro

4.4.2.5.1 CounterN

Path	Type	Value type	Description	Format	Access
..CounterN	data	JSON	See members below	JSON Object	ro
.. CounterN/Name	data	string	Counter name	“OB01_ObjectState_01”	ro
.. CounterN/Type	data	string	Counter type, enum		
.. CounterN/Value	data	float	Current counter value		ro
.. CounterN/Unit	data	string	Counter unit	“mm/s”	ro
.. CounterN/State	data	string	Current object state, enum	“Elapsed”, “”[TBD]	ro
.. CounterN/Limit	data	Int	Limit, after which an Alarm will be triggered		

4.4.2.6 Inputs

Path	Type	Value type	Description	Format	Access
..Inputs	data	JSON	See members below	See below	ro
..Inputs/Dynamic	data	JSON	See members below	See 4.4.2.7.14.4.2.6.1	ro
..Inputs/Analog	data	JSON	See members below	See 4.4.2.7.14.4.2.6.1	ro
..Inputs/External	data	JSON	See members below	See 4.4.2.7.14.4.2.6.1	ro

4.4.2.6.1 InputN

Path	Type	Value type	Description	Format	Access
..InputN	data	JSON	See members below	JSON	ro
..InputN/Name	data	string	Input's name	S1 Motor non drive end	ro
..InputN/Type	data	string	Input's type	"dynamic" „analog" "external"	ro
..InputN/Id	data	string	Input's ID	0..n	ro
..InputN/Unit	data	string	Input's unit	"m/s²"	ro
..InputN/Value	Data	Float	Value of the input	float	rw*

* Value node exists only for the "external" inputs and has r/w access

4.4.2.7 Variants

Path	Type	Value type	Description	Format	Access
..Variants	data	JSON	See members below	See below	ro
..Variants/ActiveVariant	data	JSON	The currently active variant	0-31	rw
.. Variants/VariantN	data	JSON	See members below	See 4.4.2.7.1	Ro

4.4.2.7.1 VariantN

Path	Type	Value type	Description	Format	Access
..VariantN	data	JSON	See members below	See below	ro
..VariantN/Id	data	int	The variant ID in the device	0-31	ro
.. VariantN/ActivatedObjects	data	list	List of the activated objects in this variant	[0,4,5,6]	Ro

4.4.2.8 Alarms

Path	Type	Value type	Description	Format	Access
..Alarms	structure	-	Parent of Digital and Analog alarms data	See below	ro
..Alarms/Digital	data	JSON	Digital alarms data	See 4.4.2.8.1	Ro
..Alarms/Analogue	data	JSON	Analogue alarms data	See 4.4.2.8.2	Ro

4.4.2.8.1 Digital alarms

Path	Type	Value type	Description	Format	Access
..Alarms/Digital	data	JSON	Complete digital alarms data	See below	ro
..Alarms/Digital/Alarm01..N	data	JSON	Complete data for digital alarm N	See below	ro
..Alarms/ Digital/Alarm01..N/Name	data	string	Alarm name	“OU01_Digital_01”	ro
..Alarms/ Digital/Alarm01..N/Type	data	string	Alarm type, enum	“warning”, “damage”	ro
..Alarms/ Digital/Alarm01..N/ID	data	int	Alarm id	1	Ro
..Alarms/ Digital/Alarm01..N/Value	data	bool	Alarm state	true, false	

N is integer and depends on the device capabilities

4.4.2.8.2 Analog alarms

Path	Type	Value type	Description	Format	Access
..Alarms/Analog	data	JSON	Complete digital alarms data	See below	ro
..Alarms/Analog/Alarm01..N	data	JSON	Complete data for digital alarm N	See below	Ro
..Alarms/ Analog /Alarm01..N/Name	data	string	Alarm name	“OU01_Digital_01”	ro
..Alarms/ Analog /Alarm01..N/ID	data	int	Alarm id	1	ro
..Alarms/ Analog /Alarm01..N/Value	data	float	Current alarm value	“7.124332”	ro
..Alarms/ Analog /Alarm01..N/Unit	data	string	Alarm value unit	“km/h”	ro

N is integer and depends on the device capabilities

4.4.2.9 RawData

Use this node to perform raw data recording and to retrieve the recorded data. The recorded data will be delivered in a BLOB (binary large object) with encoded data.

The recording can only be started if the VSE device's system mode is "Supervise". Any change of the system mode while recording raw data will stop the recording. Values which have been recorded so far will be available. The amount of available data can be retrieved by the blob elements.

Recording is always done for a certain amount of time (e.g. 3000 ms). The caller can wait for the recording to be finished, or he can wait for the stateChanged event to be notified about the completion of the recording.

When a recording is stopped explicitly with a call to ../RawData/stopRecording, then all values which have been recorded so far will stay available to be retrieved.

Raw data can only be recorded for one sensor at a time. However, the recorded data will be kept separately for every sensor.

Waiting for the recording to finish is possible in different ways:

4.4.2.9.1.1 A) Explicitly wait for the recording to finish

call ../RawData/startRecording(<sensor>,<samplerate>,<duration>,true)

The service response will not be sent before the recording finished. You could e.g. use the `await` method in C# to wait for the response to a `client.SendAsync()`.

4.4.2.9.1.2 B) Explicitly wait for the recording to finish via a task

call ../RawData/startRecording(<sensor>,<samplerate>,<duration>,true)

The service response will not be sent before the recording finished.

Do not wait with `await`, but assign the result of `client.SendAsync()` to a `Task` and wait for the task to finish, in which way ever appropriate.

4.4.2.9.1.3 C) Start the recording and poll the state

call ../RawData/startRecording(<sensor>,<samplerate>,<duration>,false)

then check with ../RawData/state whether the recording is running or not.

4.4.2.9.1.4 D) Start the recording and wait for an event

subscribe to the event ../RawData/stateChanged

call ../RawData/startRecording(<sensor>,<samplerate>,<duration>,false)

then wait until the event is sent.

Please note that the event notifies every change, therefore you will also get an event when the recording is started!

Path	Type	Value type	Description	Format	Access
..RawData/startRecording	service	JSON	Starts the raw data recording process	See below	ro
..RawData/stopRecording	service	JSON	Stops the raw data recording process	See below	ro

..RawData/state	data	string	“started” “stopped”		ro
..RawData/stateChanged	event	-	Event will be sent when the state changed		
..RawData/data	struct	-			ro
..RawData/data/sensorN[(1)]	blob	Base64 - encoded JSON	JSON block, containing the raw data and metadata	See below	ro
RawData/data/sensorN/size	data	int	Size of the current BLOB		ro
RawData/data/sensorN/chunksize	data	int	Size of the current BLOB		ro
RawData/data/sensorN/recordingStartTimeUTC	data	string	Start time of the recording		ro
RawData/data/sensorN/lastRecordingResult	data	string	Result of last recording (empty string if everything went OK, otherwise the error which occurred)		ro
RawData/data/sensorN/clear	service		To clear the BLOB data	No parameters	ro
RawData/data/sensorN/getdata	service	JSON	Get the BLOB	See below	

[(1)] *N can be 1..4 for the currently existing VSE models*

4.4.2.9.2 Service description

4.4.2.9.2.1 startRecording

Key	Value type	Value range	Description	Optional
sensor	int	1..4	Sensor id, for which the recording to be started	no
sampleRate	int	50000, 100000	Sample rate during raw data recording	no
duration	int	100..	Duration of the recording, in ms	no
waitUntilFinished	bool	true, false	If true, the request blocks until the raw data recording has finished Default value is “false”	yes

Success or error will be signaled in the response code.

4.4.2.9.2.2 stopRecording

This service needs no parameter.

Success or error will be signaled in the response code.

4.4.2.9.3 BLOB description

The BLOB supports only the profile defined elements size, chunksize and data.

When retrieving the recorded data with e.g. ../RawData/data/sensor1/getdata, then a JSON structure will be returned which contains these elements:

Key	Value type	Description
count	int	The number of returned values
`values`	string	The recorded values (encoded)
`recordingStartTimeUTC`	string	The start time (UTC) as string
`valueEncoding`	string	The encoding type of the `values` field. By now, this is always “Base64String”. Since more encodings may be added over time, you always should check the content of this field when decoding data.
`recordingResult`	string	The result of the last recording (empty string if everything went OK, otherwise the error which occurred).

An example to decode the values in C#:

```
string blobValues = // initialized with the string delivered by service /getdata (->data.values)
int valuesCount =  // initialized with the value delivered by service /getdata (->data.count)

var memStream = new MemoryStream(Convert.FromBase64String(jsonBlob.Data.Values));
BinaryReader reader = new BinaryReader(memStream);

var values = new List<float>();
for (int i=0; i< valuesCount; i++)
{
    values.Add(reader.ReadSingle());
}
```

4.4.2.9.4 Known limitations for raw data recording

- UDP monitoring is so far not supported by the underlying VSE driver.
- Sample rates for raw data monitoring are limited to 50000 and 100000 samples/sec by the underlying VSE driver. The additional sample rates supported by the VSE are not available.